

Python Application Programming

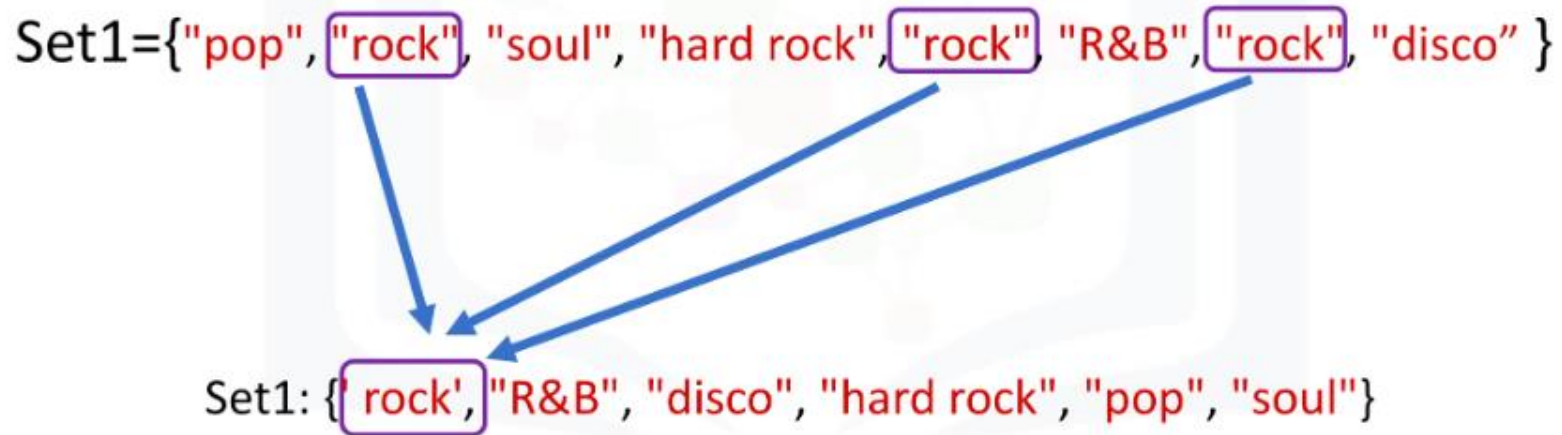
Anitya Kumar Gupta

Sets

A set is a unique collection of objects in Python.

You can denote a set with a pair of curly brackets {}.

Python will automatically remove duplicate items



Converting Lists to Sets

```
album_list = [ "Michael Jackson", "Thriller", 1982, "00:42:19", \
               "Pop, Rock, R&B", 46.0, 65, "30-Nov-82", None, 10.0]
album_set = set(album_list)
album_set
```

```
[2]: {'00:42:19',
      10.0,
      1982,
      '30-Nov-82',
      46.0,
      65,
      'Michael Jackson',
      None,
      'Pop, Rock, R&B',
      'Thriller'}
```

Different Operations in sets

We can add an element to a set using the `add()` method:

```
[5]: # Add element to set
```

```
A.add("NSYNC")
```

```
A
```

```
[5]: {'AC/DC', 'Back in Black', 'NSYNC', 'Thriller'}
```

If we add the same element twice, nothing will happen as there can be no duplicates in a set:

```
[6]: # Try to add duplicate element to the set
```

```
A.add("NSYNC")
```

```
A
```

```
[6]: {'AC/DC', 'Back in Black', 'NSYNC', 'Thriller'}
```

We can remove an item from a set using the `remove` method:

```
[7]: # Remove the element from set
```

```
A.remove("NSYNC")
```

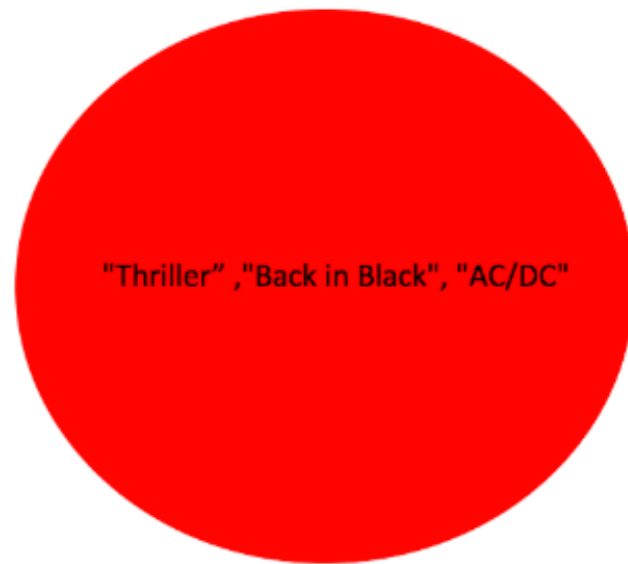
```
A
```

```
[7]: {'AC/DC', 'Back in Black', 'Thriller'}
```

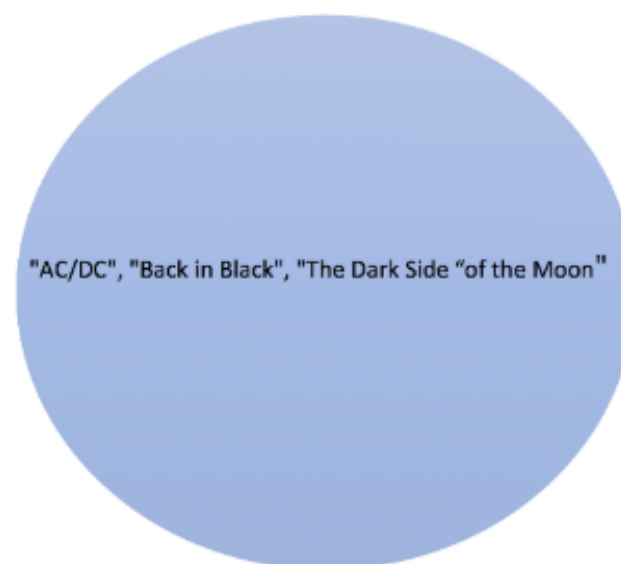
Sets Mathematical Operations

[9]: *# Sample Sets*

```
album_set1 = set(["Thriller", 'AC/DC', 'Back in Black'])  
album_set2 = set(["AC/DC", "Back in Black", "The Dark Side of the Moon"])
```



album_list1



album_list2

```
[11]: # Find the intersections
```

```
intersection = album_set1 & album_set2  
intersection
```

```
[11]: {'AC/DC', 'Back in Black'}
```

You can find all the elements that are only contained in `album_set1` using the `difference` method:

```
[12]: # Find the difference in set1 but not set2
```

```
album_set1.difference(album_set2)
```

```
[12]: {'Thriller'}
```

The union is given by:

```
[15]: # Find the union of two sets
```

```
album_set1.union(album_set2)
```

```
[15]: {'AC/DC', 'Back in Black', 'The Dark Side of the Moon', 'Thriller'}
```

And you can check if a set is a superset or subset of another set, respectively, like this:

```
[16]: # Check if superset
```

```
set(album_set1).issuperset(album_set2)
```

```
[16]: False
```

```
[17]: # Check if subset
```

```
set(album_set2).issubset(album_set1)
```

```
[17]: False
```

Different Types of Operators

- 1.Arithmetic Operators
- 2.Comparison Operators
- 3.Assignment Operators
- 4.Logical Operators
- 5.Bitwise Operators
- 6.Membership Operators
- 7.Identity Operators

Arithmetic Operators

Arithmetic operators are used to perform basic mathematical operations such as addition, subtraction, multiplication, and division.

Operator	Description	Syntax
+	Addition	$a + b$
-	Subtraction	$a - b$
*	Multiplication	$a * b$
/	Division	a / b
%	Modules	$a \% b$
**	Exponentiation	$a ** b$
//	Floor Division	$a // b$

Comparison Operator

Comparison operators are used to compare two values and return a boolean result (True or False).

Operator	Description	Syntax
==	Equal to	a == b
!=	Not equal to	a != b
>	Greater than	a > b
<	Less than	a < b
>=	Greater than or equal to	a >= b
<=	Less than or equal to	a <= b

Assignment Operator

Assignment operators are used to assign values to variables.

Operator	Description	Syntax
=	Assign	a = b
+=	Add and assign	a += b
-=	Subtract and assign	a -= b
*=	Multiply and assign	a *= b
/=	Divide and assign	a /= b
%=	Modulus and assign	a %= b
**=	Exponentiation and assign	a **= b
//=	Floor division and assign	a //= b